Abhay Kiran M

261100620002

Priority inversion

```
#include <stdint.h>
#include "stm32f103_regs.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

/* ---------------------------------------------------------------- */
/* Priorities                                                       */
/* ---------------------------------------------------------------- */
#define PRIO_LOW    (tskIDLE_PRIORITY + 1)
#define PRIO_MED    (tskIDLE_PRIORITY + 2)
#define PRIO_HIGH   (tskIDLE_PRIORITY + 3)

/* ---------------------------------------------------------------- */
/* Shared resource protection - BINARY SEMAPHORE, not a mutex       */
/* A mutex would silently fix the inversion via priority inheritance.  */
/* ---------------------------------------------------------------- */
static SemaphoreHandle_t xGpioABinarySem;

/* ---------------------------------------------------------------- */
/* Busy-wait: spins on SysTick for the requested milliseconds.      */
/* Does NOT call vTaskDelay, so the calling task never leaves Running. */
/* This is the mechanism that lets Task3 starve Task2.              */
/* ---------------------------------------------------------------- */
static volatile uint32_t g_ticks = 0;   /* incremented by tick hook */
```

```
void vApplicationTickHook(void)
{
   g_ticks++;
}

static void busy_wait_ms(uint32_t ms)
{
   uint32_t start = g_ticks;
   while ((g_ticks - start) < ms) {
      /* spin - deliberately consuming the CPU */
   }
}

/* ---------------------------------------------------------------- */
/* GPIO helpers                                                     */
/* ---------------------------------------------------------------- */
static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
   volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
   uint32_t shift = (pin % 8U) * 4U;
   *cr &= ~(0xFU << shift);
   *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

/* ---------------------------------------------------------------- */
/* Task1 - HIGH priority - blinks PA0 - needs the semaphore         */
/* ---------------------------------------------------------------- */
static void task1_high(void *pvParameters)
{
```

```
    (void)pvParameters;
    /* give Task2 a head-start so it can acquire the semaphore first */
    vTaskDelay(pdMS_TO_TICKS(10));

    for (;;) {
        /* --- WAIT for the shared GPIO-A semaphore --- */
        xSemaphoreTake(xGpioABinarySem, portMAX_DELAY);

        /* PA0 on */
        GPIOA->ODR |=  (1U << 0);
        vTaskDelay(pdMS_TO_TICKS(500));

        /* PA0 off */
        GPIOA->ODR &= ~(1U << 0);
        vTaskDelay(pdMS_TO_TICKS(500));

        xSemaphoreGive(xGpioABinarySem);
    }
}

/* ------------------------------------------------------------------ */
/* Task2 - LOW priority - blinks PA1 - needs the semaphore            */
/* ------------------------------------------------------------------ */
static void task2_low(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        /* Task2 acquires the semaphore first (it starts before Task1
         * and Task3 because of the 10 ms head-start above).
```

```
         * Once Task3 wakes and busy-waits, Task2 cannot run to give
         * the semaphore back - trapping Task1. */
        xSemaphoreTake(xGpioABinarySem, portMAX_DELAY);

        /* PA1 on */
        GPIOA->ODR |=  (1U << 1);
        vTaskDelay(pdMS_TO_TICKS(500));

        /* PA1 off */
        GPIOA->ODR &= ~(1U << 1);
        vTaskDelay(pdMS_TO_TICKS(500));

        xSemaphoreGive(xGpioABinarySem);
    }
}

/* ---------------------------------------------------------------- */
/* Task3 - MED priority - blinks PC13 - no semaphore                */
/* Busy-waits 500 ms x 5 cycles (5 s total) then sleeps 500 ms once.  */
/* The busy-wait is what blocks Task2 from running and releasing the   */
/* semaphore that Task1 is waiting on.                              */
/* ---------------------------------------------------------------- */
static void task3_med(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        /* --- 5 busy-wait cycles of 500 ms each ---------------------- */
        for (uint32_t i = 0; i < 5; i++) {
            /* PC13 toggle (active-low, so XOR gives a visible blink) */
```

```
            GPIOC->ODR ^= (1U << 13);

            /* BUSY-WAIT: does NOT yield the CPU - this is the key.
             * While spinning here, Task2 (which holds the semaphore)
             * cannot run because Task3 (MED) outranks Task2 (LOW).
             * Task1 (HIGH) is blocked on the semaphore.
             * => MED is blocking HIGH through LOW: inversion. */
            busy_wait_ms(500);
        }

        /* --- one cooperative sleep to eventually let things flow --- */
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* ------------------------------------------------------------------ */
/* main                                                               */
/* ------------------------------------------------------------------ */
int main(void)
{
    /* enable clocks for GPIOA and GPIOC */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;

    gpio_output_init(GPIOA, 0);   /* PA0 - Task1 LED */
    gpio_output_init(GPIOA, 1);   /* PA1 - Task2 LED */
    gpio_output_init(GPIOC, 13);  /* PC13 - Task3 onboard LED */

    /* BINARY semaphore - deliberately not a mutex */
    xGpioABinarySem = xSemaphoreCreateBinary();
```

```
    xSemaphoreGive(xGpioABinarySem);   /* start it available */

    xTaskCreate(task1_high,      "T1_HIGH",      configMINIMAL_STACK_SIZE,      NULL,
PRIO_HIGH, NULL);
    xTaskCreate(task2_low,       "T2_LOW",       configMINIMAL_STACK_SIZE,    NULL,
PRIO_LOW,  NULL);
    xTaskCreate(task3_med,       "T3_MED",       configMINIMAL_STACK_SIZE,    NULL,
PRIO_MED,  NULL);

    vTaskStartScheduler();

    for (;;) {}
}

void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask; (void)pcTaskName;
    for (;;) {}
}
```

Shared led no-sync

```
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
```

```
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void task1_toggle_1hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        GPIOC->ODR ^= (1U << 13);          /* on */
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);          /* off */
        vTaskDelay(pdMS_TO_TICKS(1000));
        /* one full loop iteration = one complete blink, 2000 ms total */
    }
}

static void task2_toggle_2hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        GPIOC->ODR ^= (1U << 13);          /* on */
        vTaskDelay(pdMS_TO_TICKS(500));
        GPIOC->ODR ^= (1U << 13);          /* off */
        vTaskDelay(pdMS_TO_TICKS(500));
        /* one full loop iteration = one complete blink, 1000 ms total */
    }
}
```

```
int main(void)
{
   RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
   gpio_output_init(GPIOC, 13);

   xTaskCreate(task1_toggle_1hz, "task1", configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 1, NULL);
   xTaskCreate(task2_toggle_2hz, "task2", configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 1, NULL);

   vTaskStartScheduler();

   for (;;) {
      /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
   }
}

/* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2
*/
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
   (void)xTask;
   (void)pcTaskName;
   for (;;) {
   }
}
```

Shared LED mutex

```
#include <stdint.h>
```

```
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

static SemaphoreHandle_t xLedMutex;

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void task1_toggle_1hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        xSemaphoreTake(xLedMutex, portMAX_DELAY);
        GPIOC->ODR ^= (1U << 13);          /* on */
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);          /* off */
        vTaskDelay(pdMS_TO_TICKS(1000));
        xSemaphoreGive(xLedMutex);
        /* the mutex is held for the WHOLE 2 s blink, not just each toggle */
    }
```

```
}

static void task2_toggle_2hz(void *pvParameters)
{
   (void)pvParameters;
   for (;;) {
      xSemaphoreTake(xLedMutex, portMAX_DELAY);
      GPIOC->ODR ^= (1U << 13);          /* on */
      vTaskDelay(pdMS_TO_TICKS(500));
      GPIOC->ODR ^= (1U << 13);          /* off */
      vTaskDelay(pdMS_TO_TICKS(500));
      xSemaphoreGive(xLedMutex);
      /* held for the WHOLE 1 s blink, not just each toggle */
   }
}

int main(void)
{
   RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;
   gpio_output_init(GPIOC, 13);

   xLedMutex = xSemaphoreCreateMutex();

   xTaskCreate(task1_toggle_1hz,    "task1",    configMINIMAL_STACK_SIZE,    NULL,
tskIDLE_PRIORITY + 1, NULL);
   xTaskCreate(task2_toggle_2hz,    "task2",    configMINIMAL_STACK_SIZE,    NULL,
tskIDLE_PRIORITY + 1, NULL);

   vTaskStartScheduler();
```

```
    for (;;) {
        /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
    }
}

/* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2
*/
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask;
    (void)pcTaskName;
    for (;;) {
    }
}
```

Deadlock(Synchronisation)

```
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

static SemaphoreHandle_t xLedBinary1;
static SemaphoreHandle_t xLedBinary2;

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
```

```
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void task1_toggle_1hz(void *pvParameters)
{
    (void)pvParameters;
    for (;;) {
        xSemaphoreTake(xLedBinary1, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(100));
        xSemaphoreTake(xLedBinary2, pdMS_TO_TICKS(5000));
        if(pdTRUE==0){
                xSemaphoreGive(xLedBinary1);
                vTaskDelay(pdMS_TO_TICKS(100));
                continue;

        }

        GPIOA->ODR ^= (1U << 1);
        vTaskDelay(pdMS_TO_TICKS(500));
        xSemaphoreGive(xLedBinary2);
        xSemaphoreGive(xLedBinary1);
        vTaskDelay(pdMS_TO_TICKS(100));


    }
}
```

```
static void task2_toggle_1hz(void *pvParameters)
{
   (void)pvParameters;
   for (;;) {
         for (;;) {
      xSemaphoreTake(xLedBinary2, portMAX_DELAY);
      vTaskDelay(pdMS_TO_TICKS(100));
      xSemaphoreTake(xLedBinary1, portMAX_DELAY);

      GPIOA->ODR ^= (1U << 2);          /* on */
      vTaskDelay(pdMS_TO_TICKS(500));
      xSemaphoreGive(xLedBinary1);
      xSemaphoreGive(xLedBinary2);
      vTaskDelay(pdMS_TO_TICKS(100));


         }
      }

}
int main(void)
{
   RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
   gpio_output_init(GPIOA, 1);
   gpio_output_init(GPIOA, 2);


   xLedBinary1 = xSemaphoreCreateBinary();
```

```
    xLedBinary2 = xSemaphoreCreateBinary();
    xSemaphoreGive(xLedBinary2);
    xSemaphoreGive(xLedBinary1);

    xTaskCreate(task1_toggle_1hz, "task1", configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(task2_toggle_1hz, "task2", configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 1, NULL);

    vTaskStartScheduler();

    for (;;) {
        /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
    }
}

/* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2
*/
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask;
    (void)pcTaskName;
    for (;;) {
    }
}
```

Deadlock(Synchronisation)

For below, create a deadlock and solve the deadlock

Waiting method

```
#include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

// Mutex handles for two shared resources
SemaphoreHandle_t mutexA;
SemaphoreHandle_t mutexB;

// Task 1: Acquires A first, then B
void Task1(void *pvParameters)
{
   for (;;)
   {
      // Acquire Resource A
      if (xSemaphoreTake(mutexA, portMAX_DELAY) == pdTRUE)
      {
         // Try to acquire Resource B within 100 ms
         if (xSemaphoreTake(mutexB, pdMS_TO_TICKS(100)) == pdTRUE)
         {
            // Both resources acquired successfully
            // Critical section: access Resource A and B

            // Release resources
            xSemaphoreGive(mutexB);
            xSemaphoreGive(mutexA);
```

```
            }
            else
            {
                // Resource B was unavailable
                // Release Resource A to avoid holding and waiting
                xSemaphoreGive(mutexA);
            }
        }

        // Wait before retrying
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

// Task 2: Acquires B first, then A
void Task2(void *pvParameters)
{
    for (;;)
    {
        // Acquire Resource B
        if (xSemaphoreTake(mutexB, portMAX_DELAY) == pdTRUE)
        {
            // Try to acquire Resource A within 100 ms
            if (xSemaphoreTake(mutexA, pdMS_TO_TICKS(100)) == pdTRUE)
            {
                // Both resources acquired successfully
                // Critical section: access Resource A and B

                // Release resources
```

```
            xSemaphoreGive(mutexA);
            xSemaphoreGive(mutexB);
        }
        else
        {
            // Resource A was unavailable
            // Release Resource B to avoid holding and waiting
            xSemaphoreGive(mutexB);
        }
    }

    // Different retry delay helps reduce repeated collisions
    vTaskDelay(pdMS_TO_TICKS(80));
  }
}

int main(void)
{
  // Create two mutexes
  mutexA = xSemaphoreCreateMutex();
  mutexB = xSemaphoreCreateMutex();

  // Check mutex creation
  if ((mutexA != NULL) && (mutexB != NULL))
  {
    // Create Task 1
    xTaskCreate(Task1, "Task1",
                configMINIMAL_STACK_SIZE,
                NULL, 2, NULL);
```

```
    // Create Task 2
    xTaskCreate(Task2, "Task2",
            configMINIMAL_STACK_SIZE,
            NULL, 2, NULL);

    // Start FreeRTOS scheduler
    vTaskStartScheduler();
  }

  // Reached if initialization or scheduler startup fails
  for (;;)
  {
  }
}
```

Binary mutex method

```
#include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

// Mutex handles for two shared resources
SemaphoreHandle_t mutexA;
SemaphoreHandle_t mutexB;

// Task 1
void Task1(void *pvParameters)
{
```

```
    for (;;)
    {
        // Always acquire Resource A first
        if (xSemaphoreTake(mutexA, portMAX_DELAY) == pdTRUE)
        {
            // Acquire Resource B second
            if (xSemaphoreTake(mutexB, portMAX_DELAY) == pdTRUE)
            {
                // Both resources acquired
                // Critical section: access Resource A and B

                // Release Resource B first
                xSemaphoreGive(mutexB);
            }

            // Release Resource A
            xSemaphoreGive(mutexA);
        }

        // Delay before next operation
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// Task 2
void Task2(void *pvParameters)
{
    for (;;)
    {
```

```
        // Always acquire Resource A first
        if (xSemaphoreTake(mutexA, portMAX_DELAY) == pdTRUE)
        {
            // Acquire Resource B second
            if (xSemaphoreTake(mutexB, portMAX_DELAY) == pdTRUE)
            {
                // Both resources acquired
                // Critical section: access Resource A and B

                // Release Resource B first
                xSemaphoreGive(mutexB);
            }

            // Release Resource A
            xSemaphoreGive(mutexA);
        }

        // Delay before next operation
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

int main(void)
{
    // Create two mutexes
    mutexA = xSemaphoreCreateMutex();
    mutexB = xSemaphoreCreateMutex();

    // Check mutex creation
```

```
    if ((mutexA != NULL) && (mutexB != NULL))
    {
        // Create Task 1
        xTaskCreate(Task1, "Task1",
                    configMINIMAL_STACK_SIZE,
                    NULL, 2, NULL);

        // Create Task 2
        xTaskCreate(Task2, "Task2",
                    configMINIMAL_STACK_SIZE,
                    NULL, 2, NULL);

        // Start FreeRTOS scheduler
        vTaskStartScheduler();
    }

    // Reached if initialization or scheduler startup fails
    for (;;)
    {
    }
}
```